Priorização de Testes Automatizados sob Restrição de Tempo: uma Avaliação de Heurísticas Gulosas

Andrey Lourival Andrade Garcia[1], **Igor Cecim Vilhena**[2]

[1]Centro Universitário do Estado do Pará (CESUPA)
Análise e Projeto de Algoritmos – Belém, PA, Brasil

[2]Centro Universitário do Estado do Pará (CESUPA)
Análise e Projeto de Algoritmos – Belém, PA, Brasil

Abstract. *Automated test prioritization seeks to reveal faults earlier without removing tests from a suite. This project proposes a controlled comparison of six ordering strategies: reference order, random order, shortest test first, total coverage, additional coverage, and additional coverage per unit of cost. The study connects greedy algorithm design to time and space analysis, using a small Python order-processing system as an experimental subject. The planned protocol includes generated test cases, individually seeded faults, five increasing suite sizes, and ten repetitions per configuration. Coverage and duration will guide prioritization, whereas fault information will be reserved for evaluation. APFD, APFDc, fault detection under time budgets, prioritization overhead, and auxiliary memory will be reported separately. The analysis will compare measured growth with theoretical bounds and discuss deviations caused by small instances and execution overhead. The expected contribution is a reproducible educational comparison with explicitly limited external validity, rather than a claim that a particular heuristic is universally superior. The reference matrix records the scope of source consultation and supports subsequent author verification.*

Resumo. *A priorização de testes automatizados busca revelar defeitos mais cedo sem remover testes da suíte. Este projeto propõe uma comparação controlada de seis estratégias: ordem de referência, aleatória, menor duração, cobertura total, cobertura adicional e cobertura adicional por unidade de custo. O estudo relaciona o projeto de algoritmos gulosos à análise de tempo e espaço, utilizando um pequeno sistema de pedidos em Python como objeto experimental. O protocolo prevê casos gerados, defeitos inseridos individualmente, cinco tamanhos crescentes de suíte e dez repetições por configuração. Cobertura e duração orientarão os priorizadores, enquanto as informações sobre defeitos serão reservadas à avaliação. Serão examinados APFD, APFDc, detecção sob orçamentos temporais, custo de priorização e memória auxiliar. A análise confrontará o crescimento medido com os limites teóricos e discutirá desvios decorrentes de instâncias pequenas e custos fixos. A contribuição esperada é uma comparação didática reprodutível, com validade externa explicitamente limitada, sem afirmar superioridade universal de uma heurística. A matriz bibliográfica registra o alcance da consulta às fontes e orienta sua validação pelos autores.*

Palavras-chave: priorização de testes; algoritmos gulosos; análise de complexidade; APFDc; testes de regressão.

1. Introdução

Uma suíte automatizada pode conter testes com durações e capacidades de detecção diferentes. Quando o tempo disponível é limitado, a posição de cada teste influencia o momento em que um defeito se torna observável. A priorização procura uma permutação favorável da suíte, mantendo seus elementos. Essa formulação e a comparação de critérios de cobertura aparecem em [Rothermel et al. 2001]. Seleção, minimização e priorização constituem problemas distintos, também examinados em contexto industrial por [Di Nardo et al. 2015].

Este trabalho adota o eixo E5, algoritmos gulosos, com apoio do eixo E1, análise de complexidade. Seu objetivo é comparar heurísticas simples quanto à antecipação da detecção e ao trabalho computacional necessário para produzir a ordem. O objeto é o MiniPedidos, um protótipo controlado de desconto, frete, estoque e fechamento de pedidos. Sua pequena escala permite inspecionar o procedimento, mas limita a transferência dos resultados para aplicações reais.

As perguntas de pesquisa são: **QP1**, como as estratégias diferem em detecção antecipada quando os testes têm durações distintas? **QP2**, o crescimento do custo dos priorizadores acompanha a análise assintótica? **QP3**, o benefício persiste quando se inclui o tempo de calcular a ordem? A contribuição é um protocolo reproduzível e uma análise de seus compromissos, sem reivindicar um algoritmo novo.

2. Fundamentação teórica e trabalhos relacionados

O critério total privilegia testes que cobrem muitos elementos. O adicional considera somente elementos ainda não cobertos na rodada corrente. [Li et al. 2007] analisam estratégias gulosas e de busca e mostram que objetivos diferentes podem favorecer ordens distintas. A comparação é pertinente para E5, mas antecipar cobertura não garante antecipar defeitos.

O trabalho de [Chvátal 1979] estabelece o limite harmônico $H(d)$ para o custo de uma cobertura de conjuntos obtida por escolha gulosa, com d igual ao maior tamanho de um conjunto. A razão entre novos elementos e custo inspira uma das heurísticas deste trabalho. A garantia refere-se à cobertura de custo mínimo; ela não é reivindicada para APFD, APFDc ou para a permutação completa aqui produzida.

[Elbaum et al. 2001] incorporam custos e severidades à avaliação de priorização, enquanto [Walcott et al. 2006] exploram uma abordagem genética sensível ao tempo. A revisão de [Ahmed et al. 2022] organiza métodos e métricas que consideram custo e valor. Neste projeto, a severidade é uniforme: não existem dados que permitam atribuir pesos realistas aos defeitos artificiais.

Entre os trabalhos recentes, [Li et al. 2022] reduzem o esforço do guloso adicional por aceleração e [Zhang et al. 2022] investigam atenção parcial. Ambos delimitam alternativas relevantes, mas a reprodução de aprendizado ou de AGA ampliaria o recorte. A comparação implementada utiliza critérios transparentes, cuja execução pode ser contada diretamente. A Tabela 1 sintetiza diferenças que orientam o protocolo.

Tabela 1. Comparação crítica das linhas de trabalho selecionadas.

Referência	Foco	Consequência para este projeto
Li et al. (2007)	Gulosos e busca por cobertura	Comparar total e adicional, sem tratar cobertura como detecção.
Walcott et al. (2006)	Tempo e busca genética	Medir orçamento temporal, sem reproduzir o método genético.
Li et al. (2022)	Aceleração do adicional	Contabilizar o custo da própria priorização.
Zhang et al. (2022)	Atenção parcial	Reconhecer alternativas recentes fora do recorte.
Inozemtseva e Holmes (2014)	Cobertura e eficácia	Avaliar detecção separadamente.

A família de estudos de [Elbaum et al. 2002] situa a investigação empírica de priorização. [Inozemtseva and Holmes 2014] observam que a relação entre cobertura e eficácia enfraquece ao controlar o tamanho das suítes. [Just et al. 2014] estudam a relação entre mutantes e defeitos reais, mas sua evidência não transforma qualquer conjunto artificial em substituto universal de falhas naturais. Por isso, o protocolo mantém fixos os testes de cada comparação e explicita a origem dos defeitos.

A lacuna adotada é local e didática: verificar se o benefício de reordenar funções muito curtas compensa o custo do priorizador. Os resultados de outros sistemas não resolvem essa questão para o MiniPedidos. A matriz anexa contém 12 artigos, três de 2022 e o clássico de 1979. Os níveis de consulta são discriminados, e referências acessadas apenas pelo resumo não sustentam detalhes quantitativos.

3. Metodologia planejada

3.1. Problema e algoritmos

A entrada será uma suíte T com n testes, conjuntos de linhas cobertas $C_i \subseteq U$, $m = |U|$ e durações positivas c_i. A saída deverá ser uma permutação de todos os testes. Sob orçamento B, será avaliado o maior prefixo cuja soma das durações não excede B, sem saltar testes que não caibam. As informações de detecção serão reservadas à avaliação, sem acesso pelos priorizadores.

Serão implementados referência, aleatória, menor duração, cobertura total, cobertura adicional e adicional/custo. Os dois últimos escolherão o maior ganho $|C_i \cap V|$, ou sua razão por c_i, em que V é a cobertura ainda não atingida na rodada. Ganho zero provocará reinício de V uma vez, mantendo todos os testes. Empates serão resolvidos por identificador. A validade da saída será demonstrada pelo invariante de partição entre escolhidos e restantes, com exatamente uma remoção por iteração.

Com cobertura materializada, referência e aleatória custam $O(n)$; menor duração e total, $O(n \log n)$; os adicionais, $O(n^2(m+1))$, sob acesso esperado constante aos conjuntos por hash. O espaço auxiliar será $O(n)$ nos quatro primeiros e $O(n+m)$ nos adicionais, além da entrada de até $O(nm+n)$. A fórmula de tempo inclui o caso $m = 0$. Não será reivindicada garantia de aproximação para a detecção de defeitos.

3.2. Protocolo e análise

O objeto será o MiniPedidos, protótipo Python com regras de desconto, frete, estoque e fechamento. O gerador produzirá 567 casos e serão criadas 18 versões com um defeito artificial cada. O programa-base deverá passar em todos os casos; a suíte completa deverá detectar cada alteração. Casos inválidos serão avaliados também pelo tipo de exceção esperado.

Serão construídas instâncias com $n \in \{20, 40, 80, 160, 320\}$ e dez sementes, com amostras pareadas para as seis estratégias. Cada instância será totalmente ordenada. Cobertura será coletada separadamente da medição temporal. Após aquecimento, o custo de um caso será estimado pela mediana de dez lotes de 50 chamadas; cada tempo de priorização será a mediana de três chamadas internas. A memória auxiliar será observada separadamente.

APFD avaliará posição de detecção; APFDc incorporará durações com severidade uniforme. Serão medidos defeitos detectados em frações do tempo disponível, cobertura, tempo até o primeiro defeito e tempo do priorizador. As métricas das ordens serão simuladas por soma de custos medidos, enquanto a matriz de detecção virá de execuções reais das versões defeituosas. Essa combinação não será descrita como execução de um pipeline industrial.

Serão apresentados tempo versus n, contagens de operações, médias, medianas e dispersão. Um ajuste $t(n) = an^b$ será confrontado com os limites teóricos, sem transformar regressão em prova. Como as amostras se sobrepõem e pertencem a um único objeto, a análise será descritiva. Os principais riscos são defeitos artificiais, oráculo correlacionado com o objeto, microtempos, custos fixos e limitada generalização. Código, sementes, ordens, dados e ambiente integrarão o pacote reproduzível.

Uso de IA e revisão. Esta proposta recebeu apoio das ferramentas de IA ChatGPT e Claude na busca e conferência de fontes, estruturação do texto e preparação do protocolo. A matriz registra o alcance da consulta assistida.

Referências

Ahmed, F. S., Majeed, A., Khan, T. A., and Bhatti, S. N. (2022). Value-based cost-cognizant test case prioritization for regression testing. *PLOS ONE*, 17(5):e0264972. DOI: https://doi.org/10.1371/journal.pone.0264972.

Chvátal, V. (1979). A greedy heuristic for the set-covering problem. *Mathematics of Operations Research*, 4(3):233–235. DOI: https://doi.org/10.1287/moor.4.3.233.

Di Nardo, D., Alshahwan, N., Briand, L., and Labiche, Y. (2015). Coverage-based regression test case selection, minimization and prioritization: A case study on an industrial system. *Software Testing, Verification and Reliability*, 25(4):371–396. DOI: https://doi.org/10.1002/stvr.1572.

Elbaum, S., Malishevsky, A. G., and Rothermel, G. (2001). Incorporating varying test costs and fault severities into test case prioritization. In *Proceedings of the 23rd International Conference on Software Engineering (ICSE)*, pages 329–338. IEEE. DOI: https://doi.org/10.1109/ICSE.2001.919106.

Elbaum, S., Malishevsky, A. G., and Rothermel, G. (2002). Test case prioritization: A family of empirical studies. *IEEE Transactions on Software Engineering*, 28(2):159–182. DOI: https://doi.org/10.1109/32.988497.

Inozemtseva, L. and Holmes, R. (2014). Coverage is not strongly correlated with test suite effectiveness. In *Proceedings of the 36th International Conference on Software Engineering (ICSE)*, pages 435–445. ACM. DOI: https://doi.org/10.1145/2568225.2568271.

Just, R., Jalali, D., Inozemtseva, L., Ernst, M. D., Holmes, R., and Fraser, G. (2014). Are mutants a valid substitute for real faults in software testing? In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)*, pages 654–665. ACM. DOI: https://doi.org/10.1145/2635868.2635929.

Li, F., Zhou, J., Li, Y., Hao, D., and Zhang, L. (2022). AGA: An accelerated greedy additional algorithm for test case prioritization. *IEEE Transactions on Software Engineering*, 48(12):5102–5119. DOI: https://doi.org/10.1109/TSE.2021.3137929.

Li, Z., Harman, M., and Hierons, R. M. (2007). Search algorithms for regression test case prioritization. *IEEE Transactions on Software Engineering*, 33(4):225–237. DOI: https://doi.org/10.1109/TSE.2007.38.

Rothermel, G., Untch, R. H., Chu, C., and Harrold, M. J. (2001). Prioritizing test cases for regression testing. *IEEE Transactions on Software Engineering*, 27(10):929–948. DOI: https://doi.org/10.1109/32.962562.

Walcott, K. R., Soffa, M. L., Kapfhammer, G. M., and Roos, R. S. (2006). TimeAware test suite prioritization. In *Proceedings of the International Symposium on Software Testing and Analysis (ISSTA)*, pages 1–12. ACM. DOI: https://doi.org/10.1145/1146238.1146240.

Zhang, Q., Fang, C., Sun, W., Yu, S., Xu, Y., and Liu, Y. (2022). Test case prioritization using partial attention. *Journal of Systems and Software*, 192:111419. DOI: https://doi.org/10.1016/j.jss.2022.111419.